6.5830/1 Lecture 16: Distributed Transactions

4/9/2026

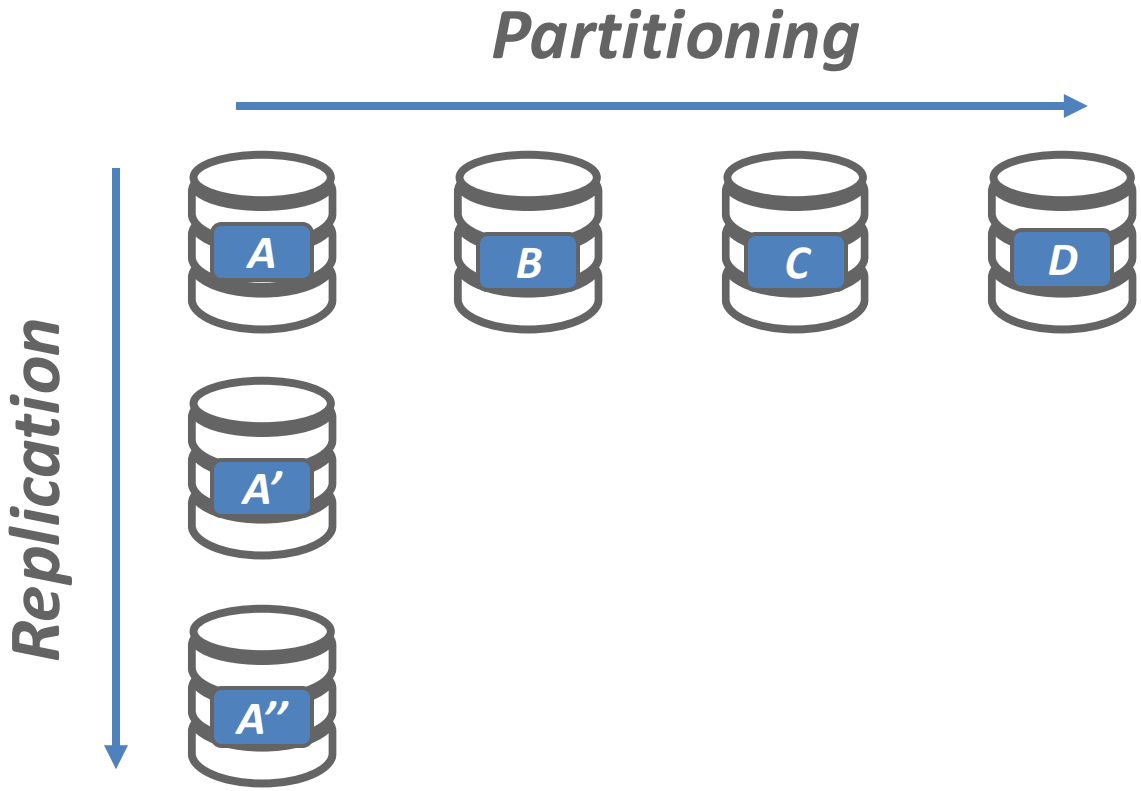
Tianyu Li

litianyu@mit.edu

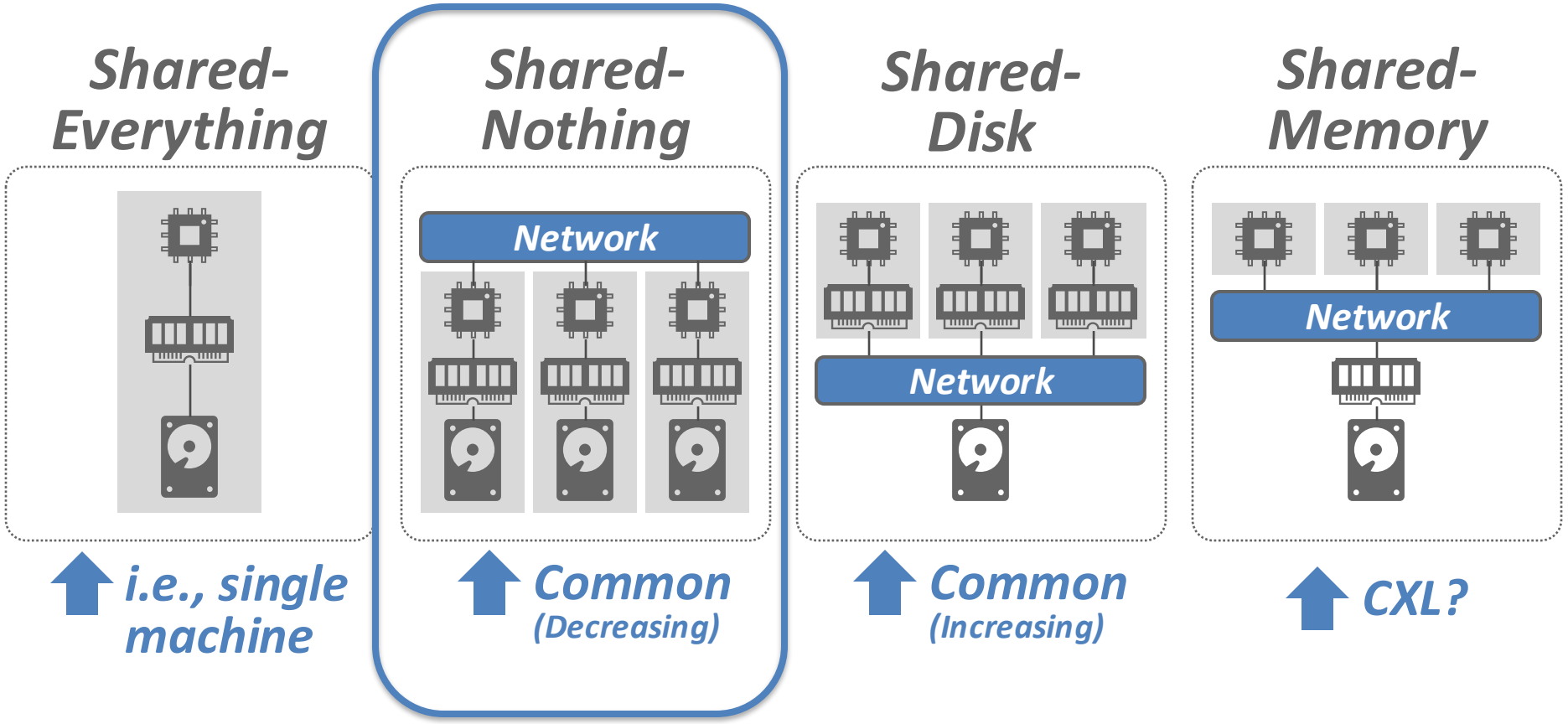
Administrivia

- Pset 3 out
- Project Meetings + midterm reports (4/14)
- Lab 3 due next week (4/16)
  - Writeup 4/21

Last Class



Distributed DBMS Architecture



Observation

- Recall that our goal is to have multiple physical nodes appear as a single logical DBMS.

- How should we support ACID transactions?
  - Example 1: reserve a rental car and an airline flight, and only commit if both are available.
  - Example 2: transfer money from bank 1 to bank 2

Distributed Transactions

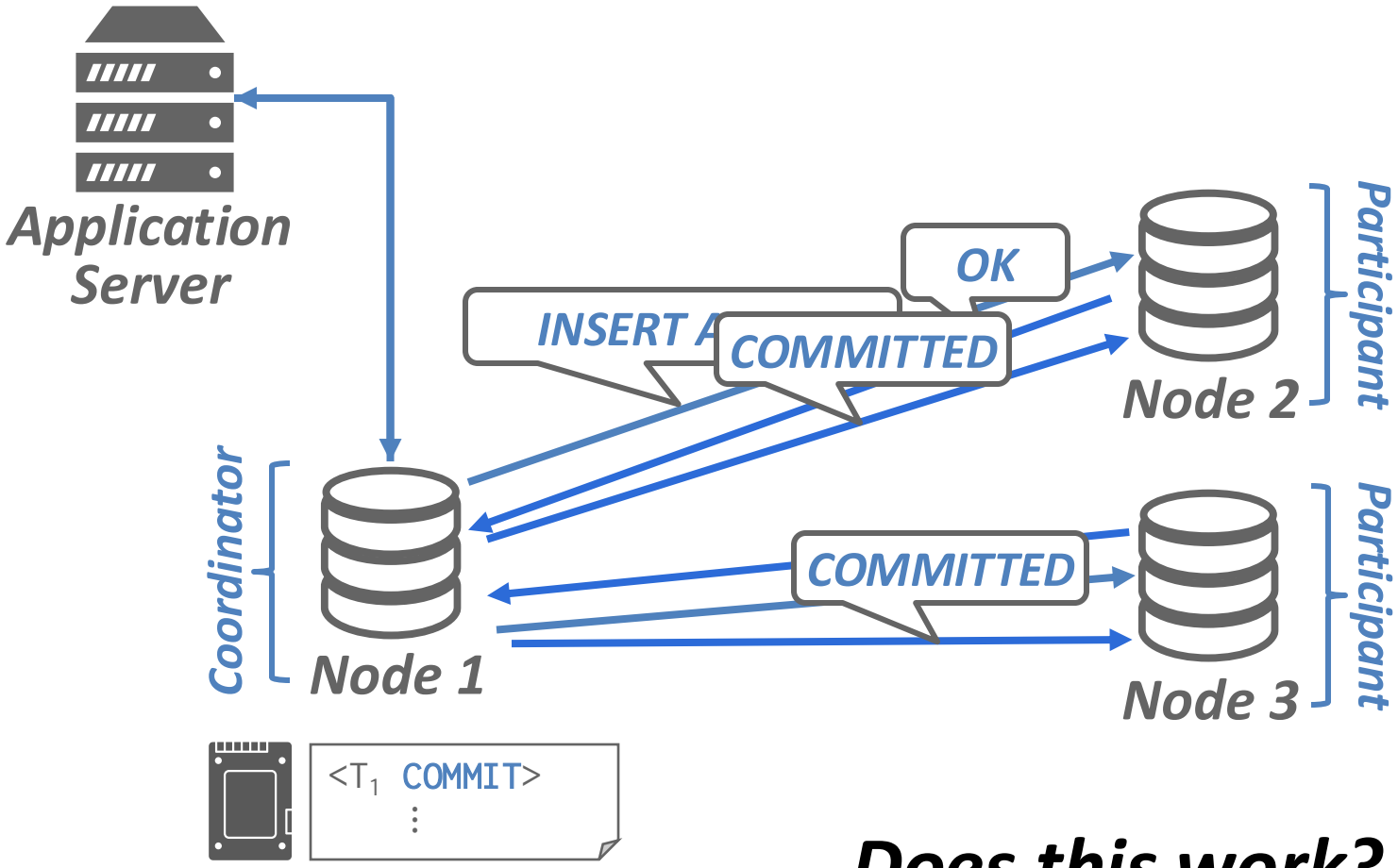
Similar to single-node transactions, two parts:

- Concurrency Control to provide isolation/serializability
  - Single-node concurrency control usually suffices with minor* tweaks
- Atomic commit to provide atomicity/durability
  - On a single node: use ARIES
  - Can we extend that as well?

Strawman

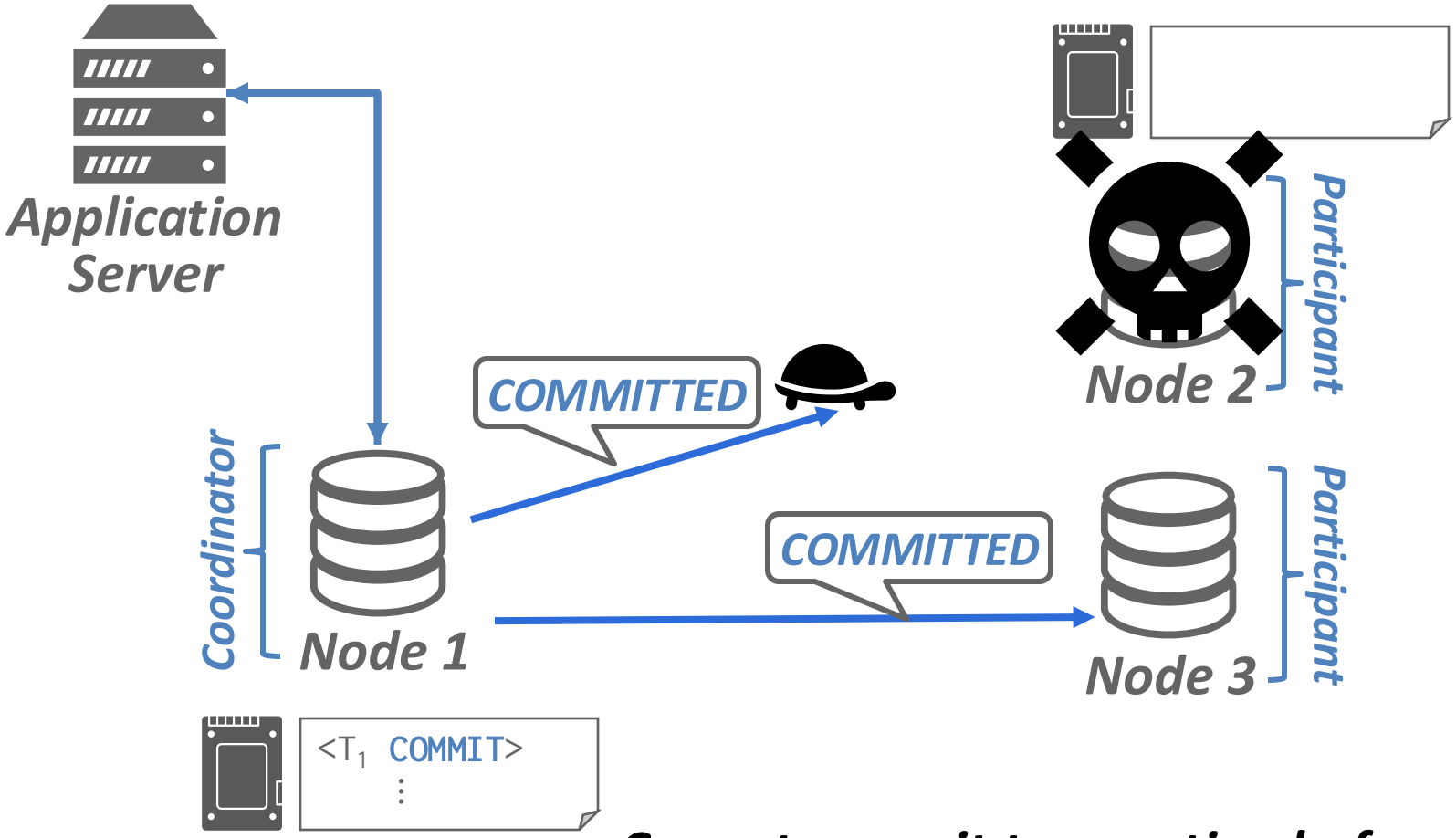
- Idea: run ARIES at each component
- One worker flushes the commit record and declare commit
- Notify everyone to release locks

Strawman



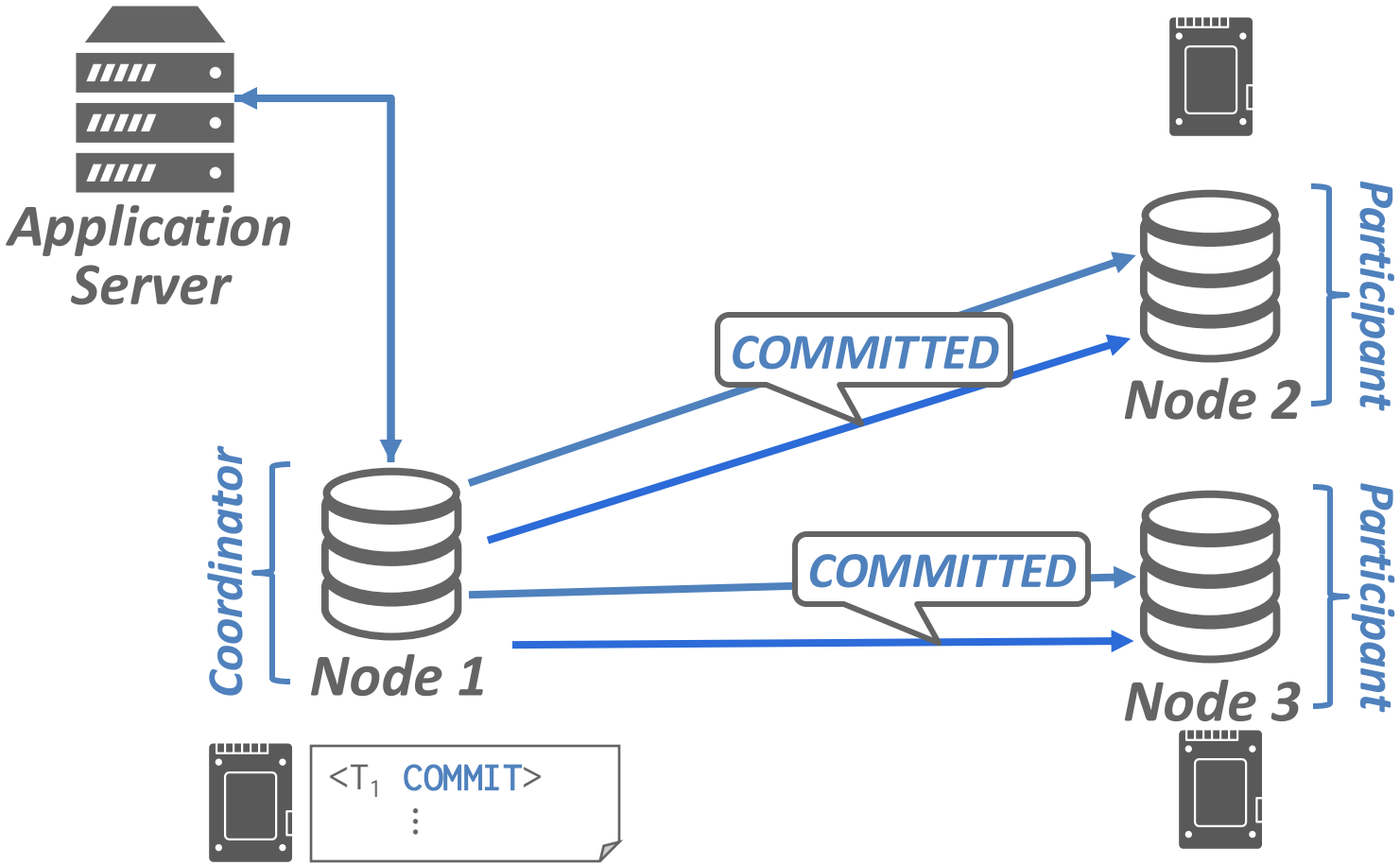
Does this work?

Strawman on Fire



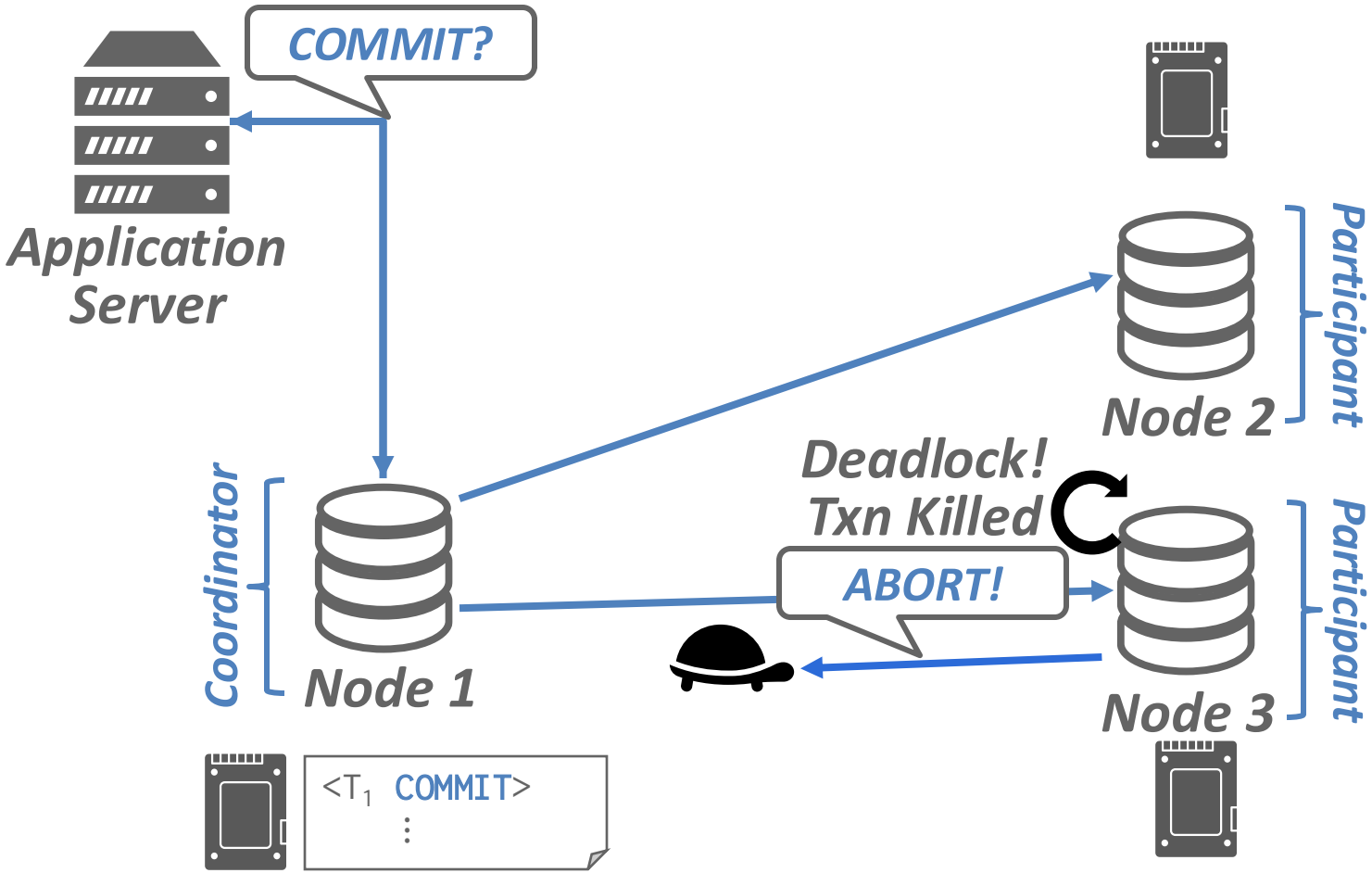
Cannot commit transaction before we know components are recoverable!

Strawman 2



Does this work?

Strawman 2 on Fire



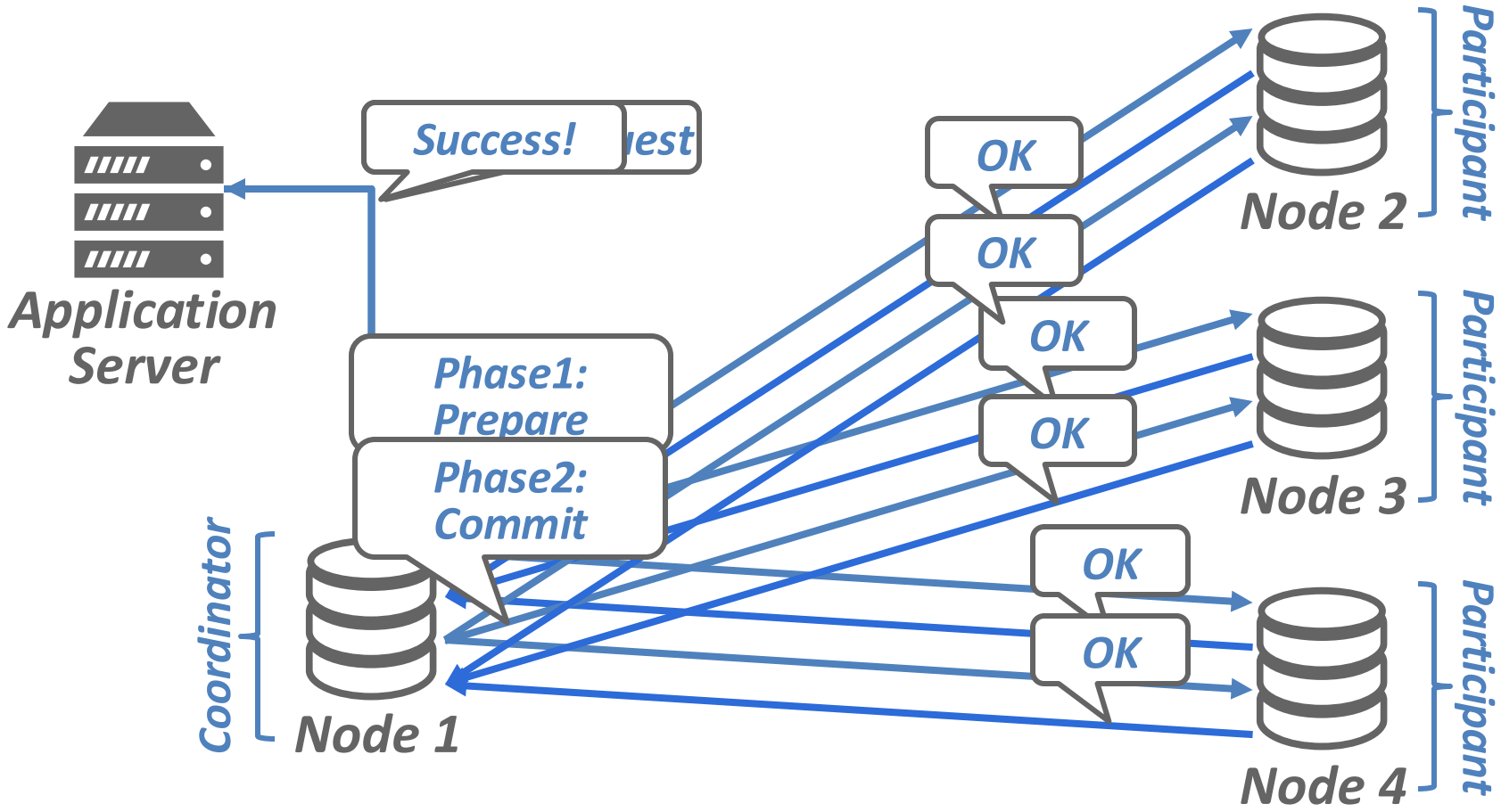
Cannot commit transaction if components can renege!

Attempt 3

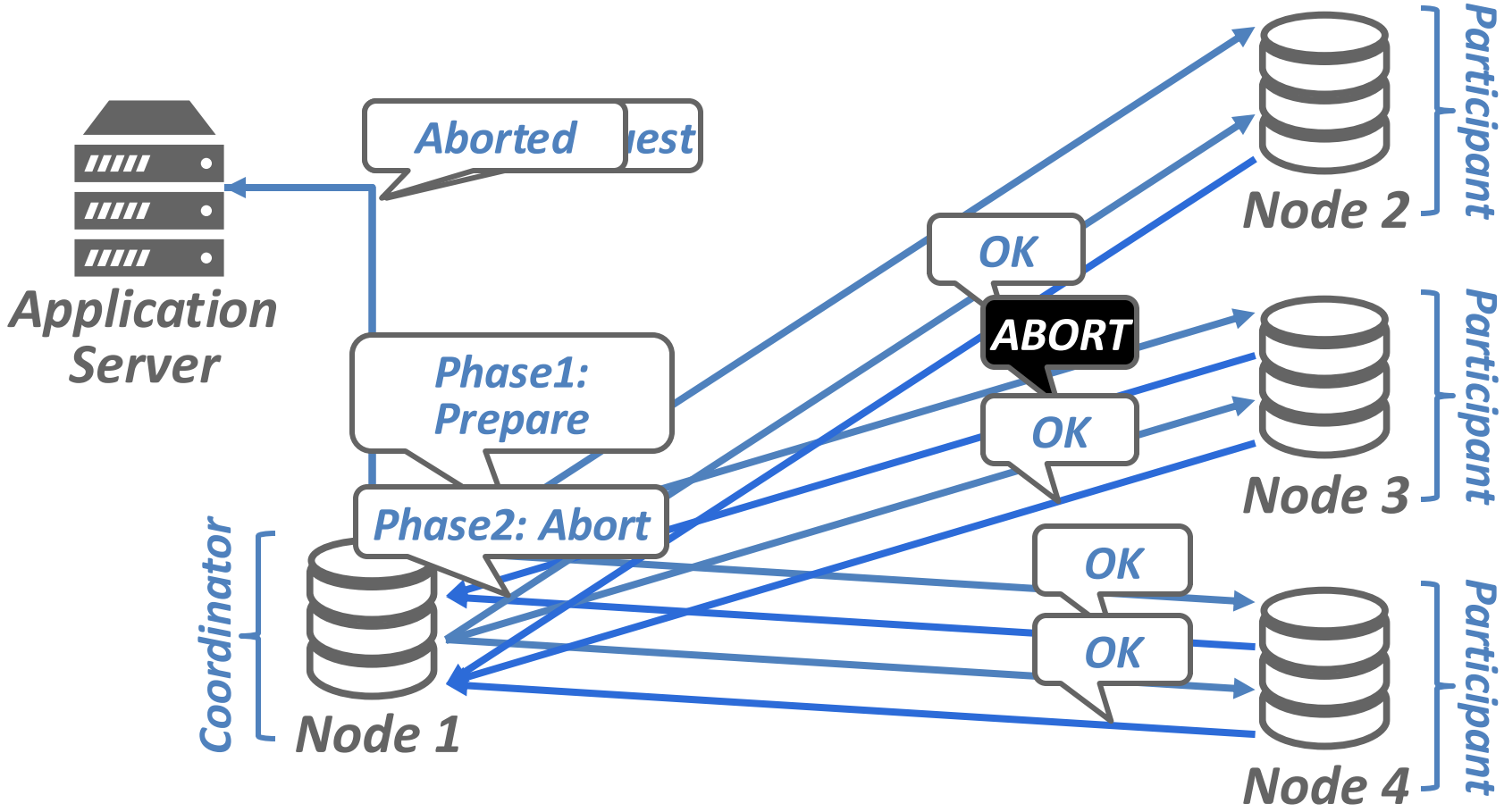
- Idea: participants locally “commit”, but wait for a round of communications to actually commit
  - Let’s call this “PREPARED”

- This is called **two-phase commit**

Attempt 3


Application Server
Success!
Phase1: Prepare
Phase2: Commit
Coordinator
Node 1
OK
OK
OK
OK
OK
OK
Node 2
Node 3
Node 4
Participant
Participant
Participant

Attempt 3



Does this work?

Yes, but…

- First lesson of distributed systems – distributed systems is hard

- We want to properly understand the intricacies through mathematical modeling rather than find out in production

Distributed Systems

- Definitions vary, but usually means a group of computers cooperating to act as a cohesive unit

- Computers communicate over a network
  - Formalization exist, but we will not talk about it

Types of Failures

- Link Failures
  - Network may delay, reorder, corrupt, drop messages
- Process Failures
  - Computers may crash or become malicious (i.e., fail-stop vs. Byzantine)
  - For databases, we usually assume fail-stop with recovery

Distributed Consensus

- Fundamental problem in distributed Systems
- Goal: reach agreement among nodes with varying starting opinions
  - Complications: failures may happen
  - Must have consensus amongst surviving nodes
- Observation: very close to what we want in a distributed commit protocol
  - However, they are not the same – failing participants cause aborts, but may not hinder consensus

System Assumptions

- Early days: synchronous network
  - Participants perform synchronized rounds of communication + work
  - Often reasonable to assume for components within a physical machine
  - Many problems are solvable in this model

Achieving Consensus

- Consensus possible with node failures in synchronous networks

- Jim Gray showed that consensus is impossible under link failures where messages may be dropped
  - i.e., the two generals problem
  - Impossibility applies even if upper bound for successful message delay exists

Achieving Consensus

- Problem: real-world network experiences failures and DBMS must be able to tolerate it

- New assumption: asynchronous network
  - No message drop
  - Timing may vary
  - Think TCP

Disaster

- Famous Result (Fischer, Lynch, Paterson): Consensus is impossible in asynchronous networks with even just 1 node failure
  - "cannot distinguish slow from dead"

- However, practical consensus algorithms are possible if we relax guarantees
  - E.g., conditional termination only if there is a "sufficiently long" window of good behavior
  - More on this later…

Back to 2PC…

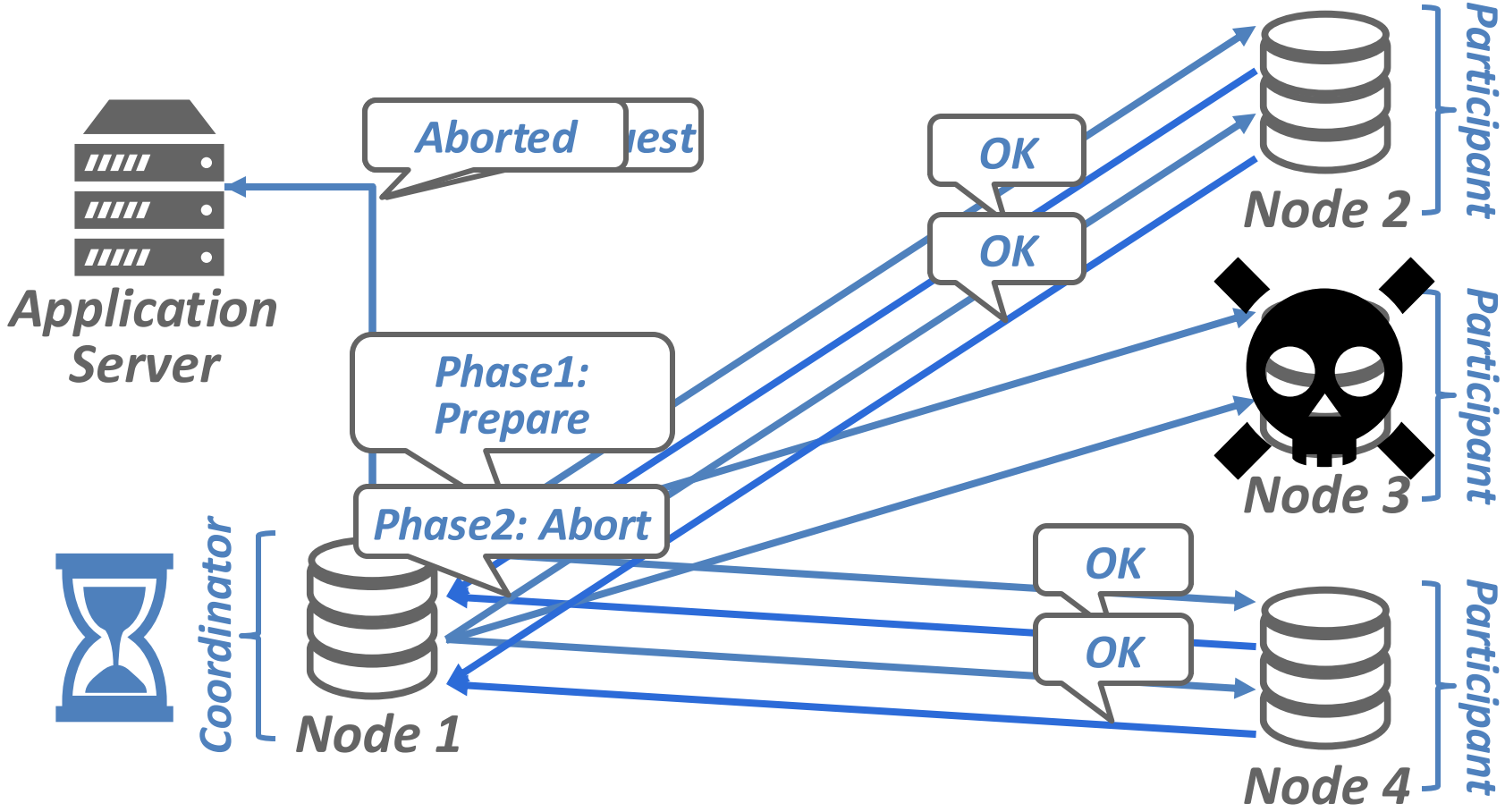
- Must be VERY careful about our assumptions and the order of operations
- Assumption: stopping failures only, asynchronous network
- Correctness condition: agreement only (for now)
  - Not sufficient for databases, more on this later

Correctness of 2PC

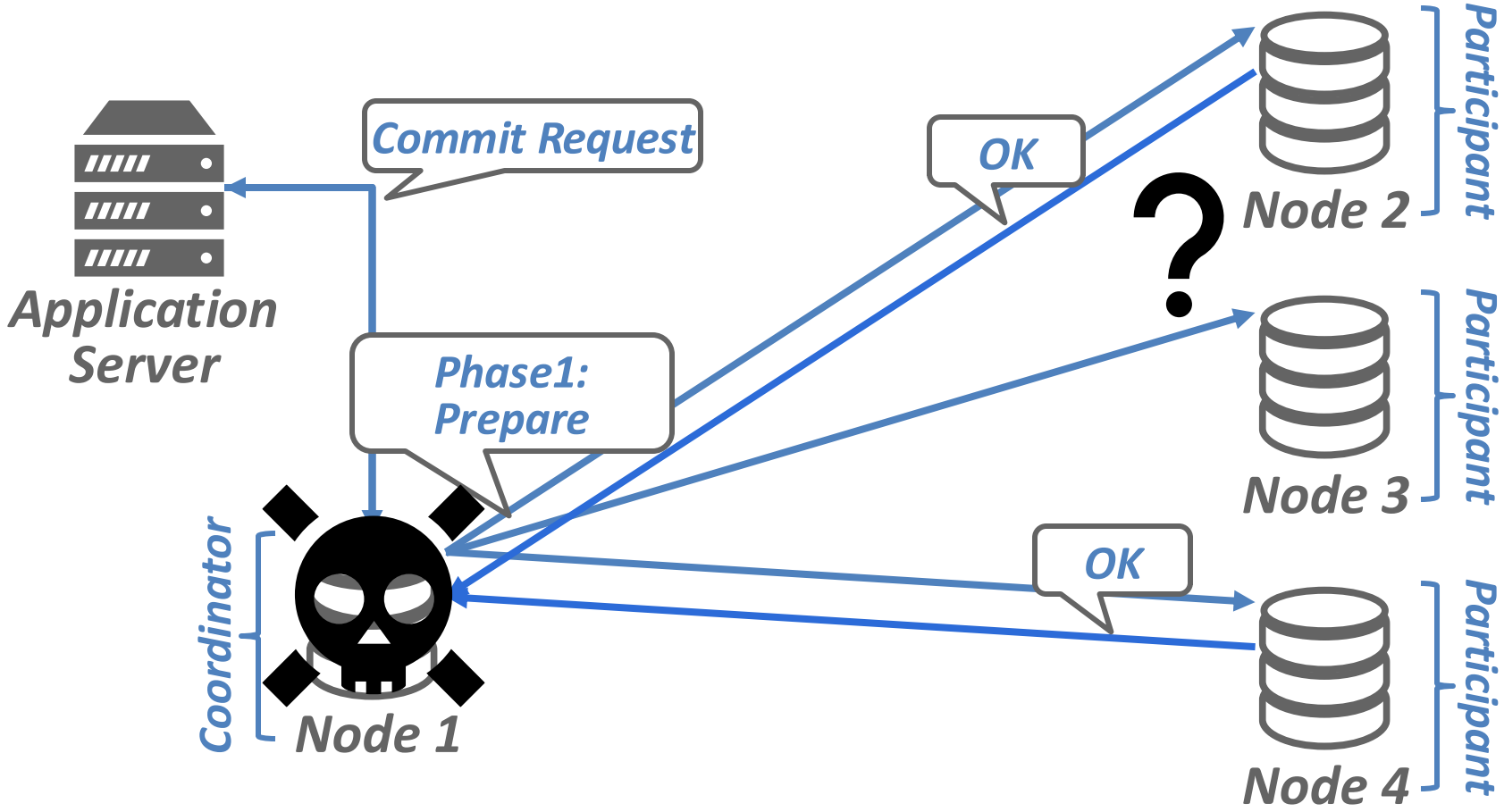
- Nodes cannot disagree
  - Once PREPARED, participants do not renege
  - Coordinator only declares commit after hearing from everyone

- Important: FLP tells us that it is impossible. Therefore, 2PC must have some flaws

Two Phase Commit: Participant Failure



Two Phase Commit: Coordinator Failure



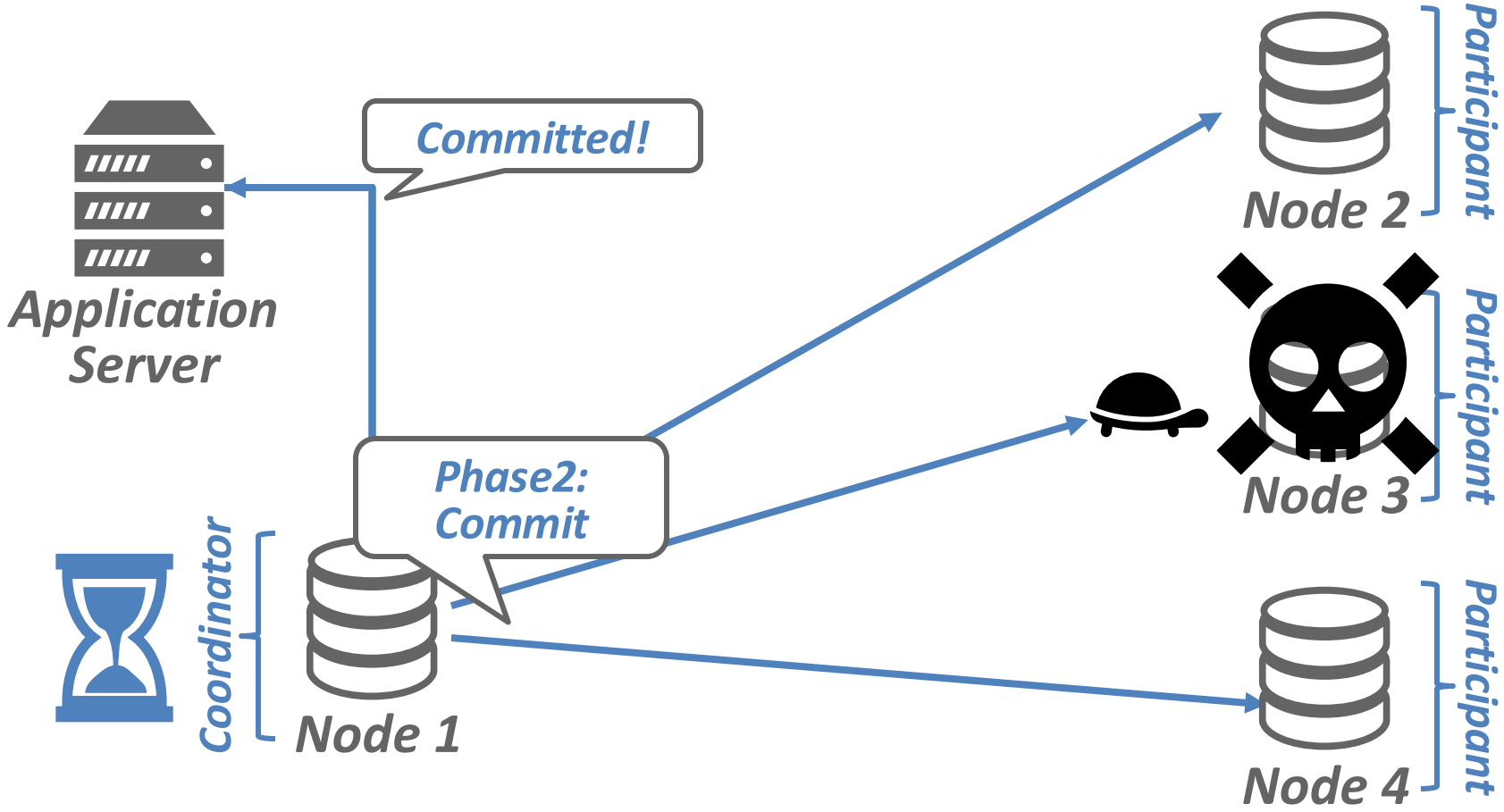
Important

- In two-phase commit, coordinator decision is atomic decision point
  - Once coordinator locally decides commit, that's point of no return
  - Some variants can report commit to clients early because of this
- Problem: if coordinator crashes, participants cannot safely make progress
  - Timeouts cannot help us

Three-Phase Commit

- Theoretical solution to allow for progress when coordinator dies
- Idea: participants go into “pre-commit” phase after voting yes to introduce timeout
  - Cool algorithm, relies on partial synchrony
- **Nobody** uses this in practice
  - Performance reasons, safety issues, better alternatives, etc.

Next, Recovery…



2PC + ARIES

- Failed database nodes must recover correctly to guarantee ACID in addition to agreement
  - Log transaction as PREPARED and flush before voting
  - On replay, re-acquire locks for PREPARED transactions
  - Ask coordinator about status of transaction

Coordinator Recovery

- Log decision and flush before sending any message
- Track progress of workers, log DONE after all workers ack decision
- On recovery, log tells us which transactions were running
  - Abort all transactions not committed

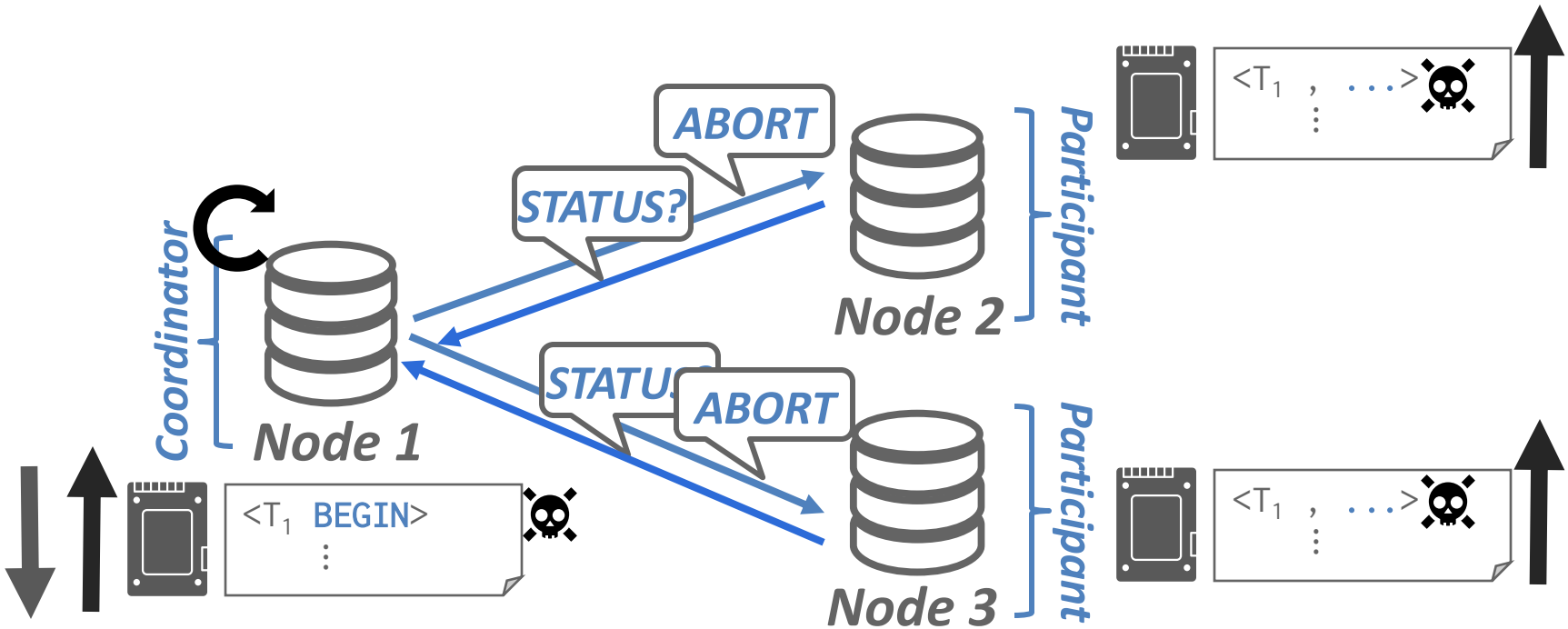
Coordinator Recovery

- Must be prepared to answer worker questions
  - Must remember any transaction not with DONE
  - Design decision to presume a decision on transactions we have no record of
  - Depending on choice, log different info
- Must co-design with participant side logic to ensure progress
  - Coordinator pushes / participant pulls
- Typical: presumed abort, participants pull

Failure Cases

Example Scenario 1: Coordinator fails before sending PREPARE

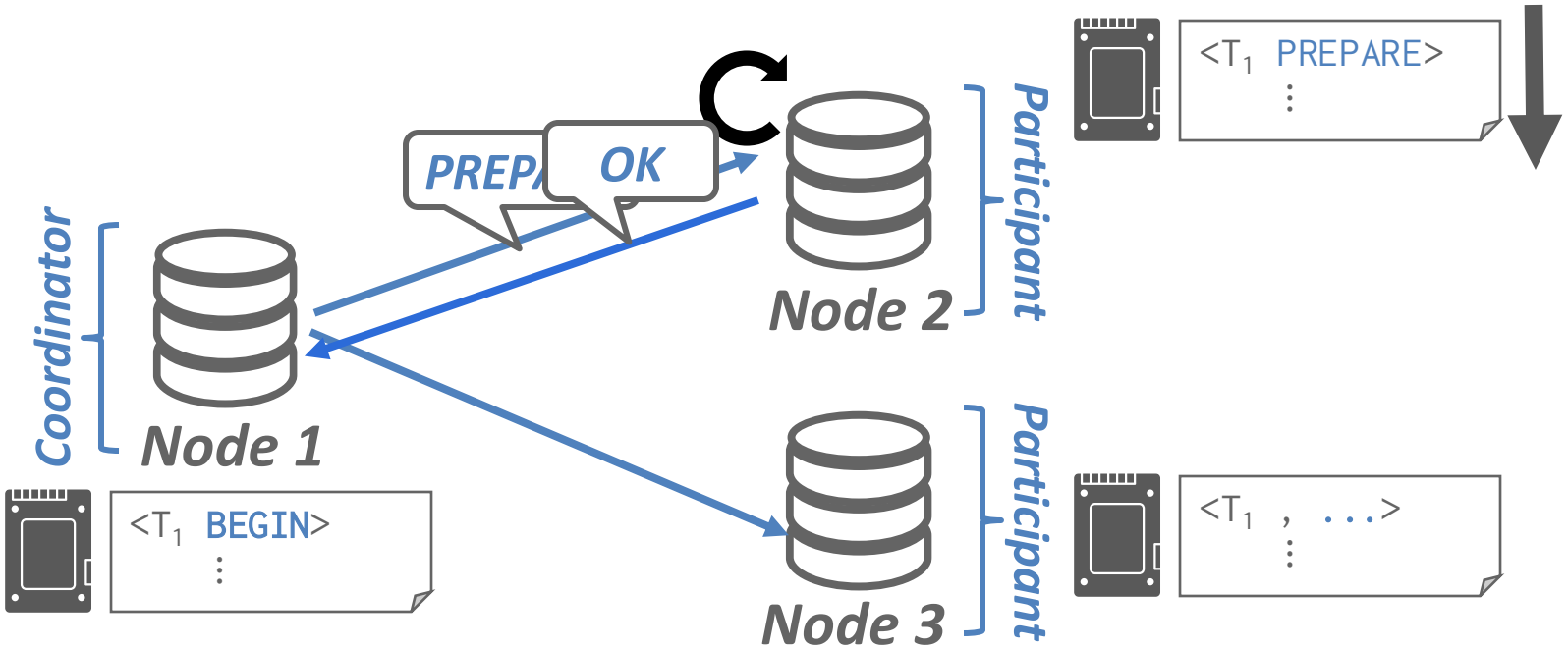
- Coordinator will recover and abort



Failure Cases

Example Scenario 2: Participant crashes after PREPARE

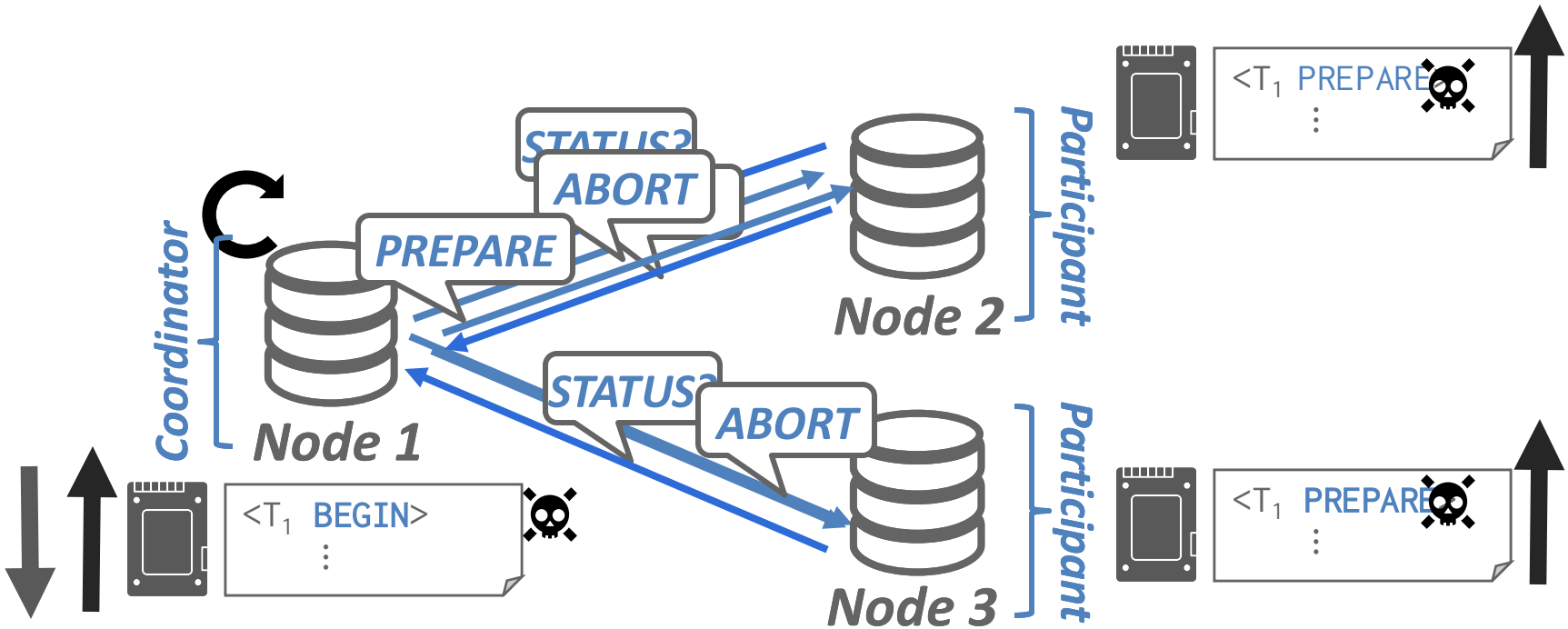
- Worker recover into PREPARE
- Outcome depends on coordinator



Failure Cases

Example Scenario 3: Coordinator crashes before decision

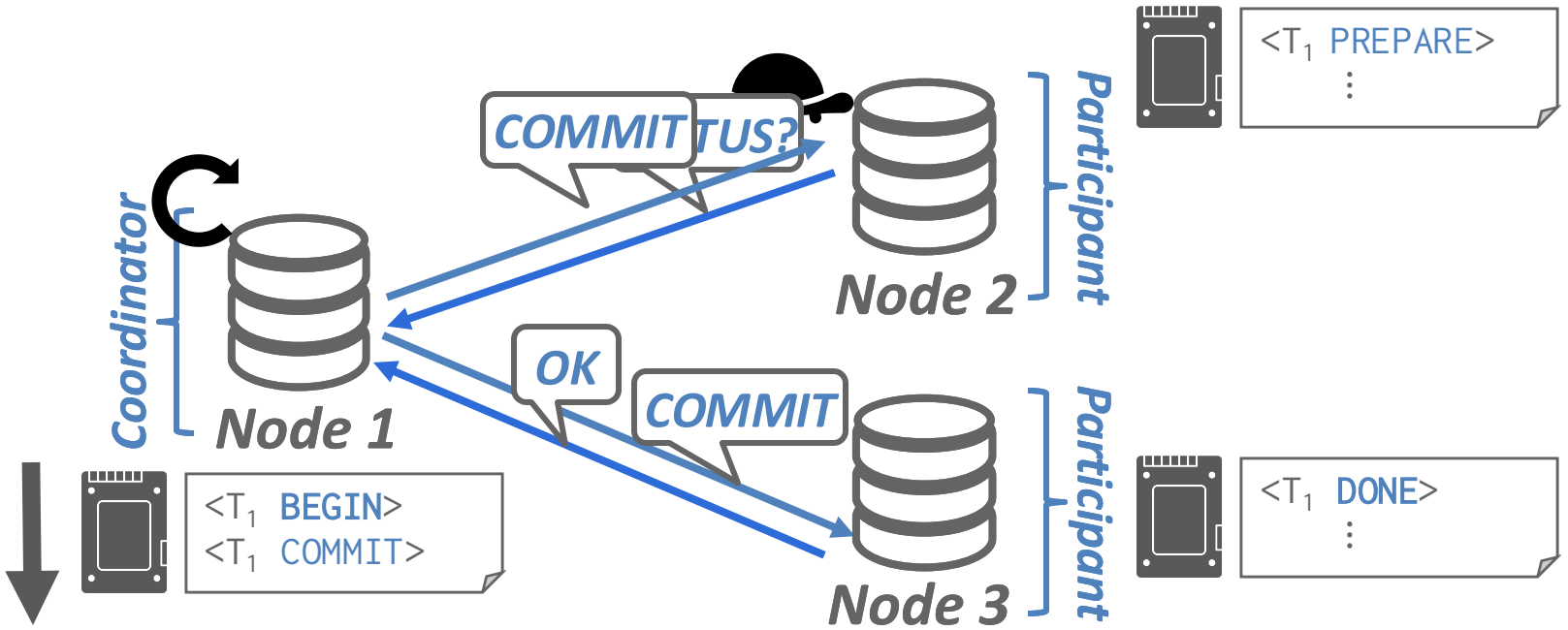
- Coordinator recover and aborts
- Workers find out next pull and aborts



Failure Cases

Example Scenario 4: Coordinator crashes after decision

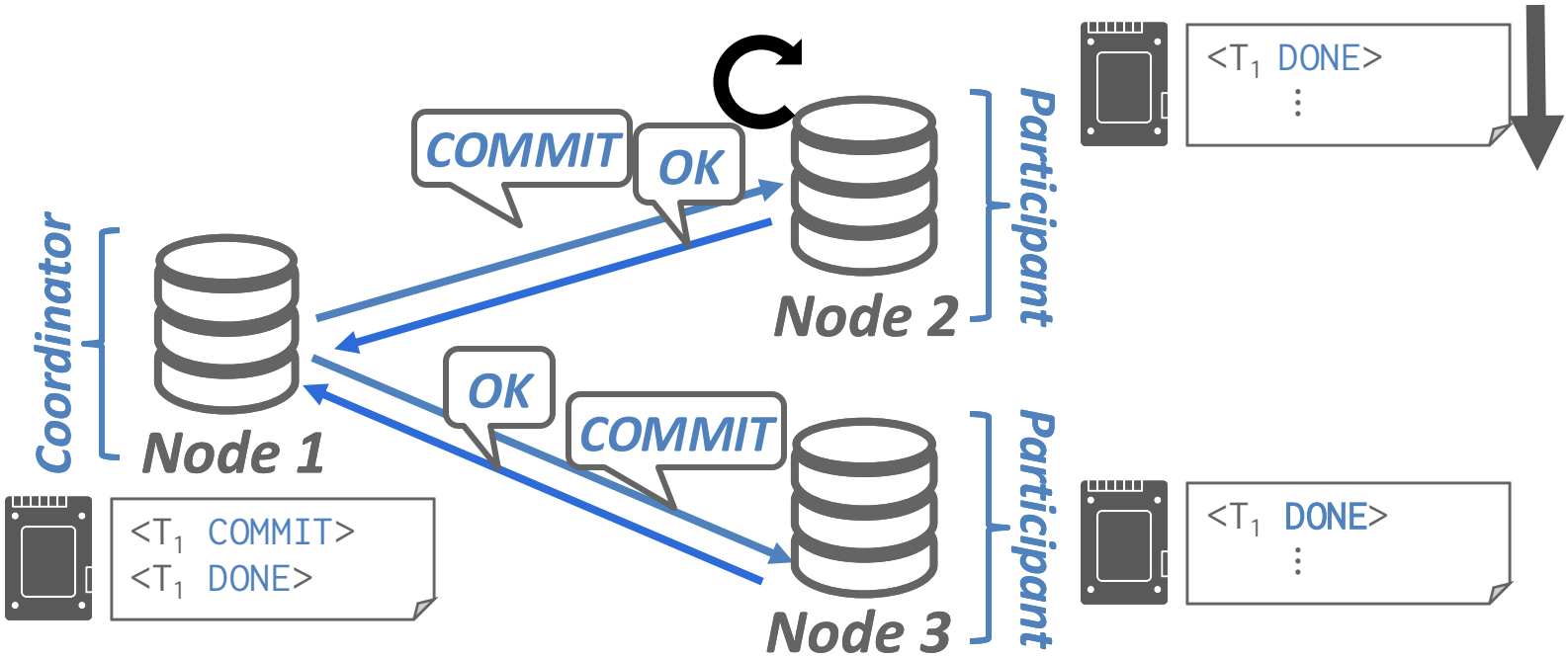
- No DONE record; coordinator remembers decision
- Workers will find out on poll



Failure Cases

Example Scenario 5: Participant crashes after committing locally but before acking

- Recovery restores state, worker should re-ack



2PC – Problems

- High overhead from synchronous logging & network roundtrips
  - Holding locks throughout!
  - Particularly disastrous over wide-area networks

- System is not fault-tolerant in the sense that it continues to function after partial failure
  - Coordinator crashes can cause the system to hang and become unavailable

Example: Google Spanner

- A rare example of geo-distributed strongly consistent transactional system
- Specialized hardware (TrueTime) + Optimized 2PC on Paxos (more on this later)

Spanner: Google's Globally-Distributed Database

James C. Corbett, Jeffrey Dean, Michael Epstein, Andrew Fikes, Christopher Frost, JJ Furman, Sanjay Ghemawat, Andrey Gubarev, Christopher Heiser, Peter Hochschild, Wilson Hsieh, Sebastian Kanthak, Eugene Kogan, Hongyi Li, Alexander Lloyd, Sergey Melnik, David Mwaura, David Nagle, Sean Quinlan, Rajesh Rao, Lindsay Rolig, Yasushi Saito, Michal Szymaniak, Christopher Taylor, Ruth Wang, Dale Woodford

Google, Inc.

Abstract

Spanner is Google's scalable, multi-version, globally-distributed, and synchronously-replicated database. It is the first system to distribute data at global scale and support externally-consistent distributed transactions. This paper describes how Spanner is structured, its feature set, the rationale underlying various design decisions, and a novel time API that exposes clock uncertainty. This API and its implementation are critical to supporting external consistency and a variety of powerful features: non-blocking reads in the past, lock-free read-only transactions, and atomic schema changes, across all of Spanner.

1 Introduction

Spanner is a scalable, globally-distributed database designed, built, and deployed at Google. At the highest level of abstraction, it is a database that shards data across many sets of Paxos [21] state machines in datacenters spread all over the world. Replication is used for global availability and geographic locality; clients automatically failover between replicas. Spanner automatically reshards data across machines as the amount of data or the number of servers changes, and it automatically migrates data across machines (even across datacenters) to balance load and in response to failures. Spanner is designed to scale up to millions of machines across hundreds of datacenters and trillions of database rows.

Applications can use Spanner for high availability, even in the face of wide-area natural disasters, by replicating their data within or even across continents. Our initial customer was F1 [35], a rewrite of Google's advertising backend. F1 uses five replicas spread across

tency over higher availability, as long as they can survive 1 or 2 datacenter failures.

Spanner's main focus is managing cross-datacenter replicated data, but we have also spent a great deal of time in designing and implementing important database features on top of our distributed-systems infrastructure. Even though many projects happily use Bigtable [9], we have also consistently received complaints from users that Bigtable can be difficult to use for some kinds of applications: those that have complex, evolving schemas, or those that want strong consistency in the presence of wide-area replication. (Similar claims have been made by other authors [37].) Many applications at Google have chosen to use Megastore [5] because of its semi-relational data model and support for synchronous replication, despite its relatively poor write throughput. As a consequence, Spanner has evolved from a Bigtable-like versioned key-value store into a temporal multi-version database. Data is stored in schematized semi-relational tables; data is versioned, and each version is automatically timestamped with its commit time; old versions of data are subject to configurable garbage-collection policies; and applications can read data at old timestamps. Spanner supports general-purpose transactions, and provides a SQL-based query language.

As a globally-distributed database, Spanner provides several interesting features. First, the replication configurations for data can be dynamically controlled at a fine grain by applications. Applications can specify constraints to control which datacenters contain which data, how far data is from its users (to control read latency), how far replicas are from each other (to control write latency), and how many replicas are maintained (to control durability, availability, and read performance). Data

Corbett et. al. Spanner: Google's Globally-Distributed Database. OSDI 2012

Google Spanner

replicas	latency (ms)			throughput (Kops/sec)		
	write	read-only transaction	snapshot read	write	read-only transaction	snapshot read
1D	9.4±.6	—	—	4.0±.3	—	—
1	14.4±1.0	1.4±.1	1.3±.1	4.1±.05	10.9±.4	13.5±.1
3	13.9±.6	1.3±.1	1.2±.1	2.2±.5	13.8±3.2	38.5±.3
5	14.4±.4	1.4±.05	1.3±.04	2.8±.3	25.3±5.2	50.0±1.1

- Throughput: many reads, but only a few thousand txns per second

Google Spanner

participants	latency (ms)	
	mean	99th percentile
1	17.0 ±1.4	75.0 ±34.9
2	24.5 ±2.5	87.6 ±35.9
5	31.5 ±6.2	104.5 ±52.2
10	30.0 ±3.7	95.6 ±25.4
25	35.5 ±5.6	100.4 ±42.7
50	42.7 ±4.1	93.7 ±22.9
100	71.4 ±7.6	131.2 ±17.6
200	150.5 ±11.0	320.3 ±35.1

- 2PC scalability is poor

- This translates to high end-to-end latency in production scenarios

operation	latency (ms)		count
	mean	std dev	
all reads	8.7	376.4	21.5B
single-site commit	72.3	112.8	31.2M
multi-site commit	103.0	52.2	32.1M

Distributed Transactions Research

- Still area of active research after decades of work

- Latest ideas: workload specialization, deterministic transactions, intelligent batching, etc.
  - We won't have time to cover them in class

Example: Google Spanner

- A rare example of geo-distributed strongly consistent transactional system
- Specialized hardware (TrueTime) + Optimized 2PC on Paxos (more on this later)

Spanner: Google's Globally-Distributed Database

James C. Corbett, Jeffrey Dean, Michael Epstein, Andrew Fikes, Christopher Frost, JJ Furman, Sanjay Ghemawat, Andrey Gubarev, Christopher Heiser, Peter Hochschild, Wilson Hsieh, Sebastian Kanthak, Eugene Kogan, Hongyi Li, Alexander Lloyd, Sergey Melnik, David Mwaura, David Nagle, Sean Quinlan, Rajesh Rao, Lindsay Rolig, Yasushi Saito, Michal Szymaniak, Christopher Taylor, Ruth Wang, Dale Woodford

Google, Inc.

Abstract

Spanner is Google's scalable, multi-version, globally-distributed, and synchronously-replicated database. It is the first system to distribute data at global scale and support externally-consistent distributed transactions. This paper describes how Spanner is structured, its feature set, the rationale underlying various design decisions, and a novel time API that exposes clock uncertainty. This API and its implementation are critical to supporting external consistency and a variety of powerful features: non-blocking reads in the past, lock-free read-only transactions, and atomic schema changes, across all of Spanner.

1 Introduction

Spanner is a scalable, globally-distributed database designed, built, and deployed at Google. At the highest level of abstraction, it is a database that shards data across many sets of Paxos [21] state machines in datacenters spread all over the world. Replication is used for global availability and geographic locality; clients automatically failover between replicas. Spanner automatically reshards data across machines as the amount of data or the number of servers changes, and it automatically migrates data across machines (even across datacenters) to balance load and in response to failures. Spanner is designed to scale up to millions of machines across hundreds of datacenters and trillions of database rows.

Applications can use Spanner for high availability, even in the face of wide-area natural disasters, by replicating their data within or even across continents. Our initial customer was F1 [35], a rewrite of Google's advertising backend. F1 uses five replicas spread across

tency over higher availability, as long as they can survive 1 or 2 datacenter failures.

Spanner's main focus is managing cross-datacenter replicated data, but we have also spent a great deal of time in designing and implementing important database features on top of our distributed-systems infrastructure. Even though many projects happily use Bigtable [9], we have also consistently received complaints from users that Bigtable can be difficult to use for some kinds of applications: those that have complex, evolving schemas, or those that want strong consistency in the presence of wide-area replication. (Similar claims have been made by other authors [37].) Many applications at Google have chosen to use Megastore [5] because of its semi-relational data model and support for synchronous replication, despite its relatively poor write throughput. As a consequence, Spanner has evolved from a Bigtable-like versioned key-value store into a temporal multi-version database. Data is stored in schematized semi-relational tables; data is versioned, and each version is automatically timestamped with its commit time; old versions of data are subject to configurable garbage-collection policies; and applications can read data at old timestamps. Spanner supports general-purpose transactions, and provides a SQL-based query language.

As a globally-distributed database, Spanner provides several interesting features. First, the replication configurations for data can be dynamically controlled at a fine grain by applications. Applications can specify constraints to control which datacenters contain which data, how far data is from its users (to control read latency), how far replicas are from each other (to control write latency), and how many replicas are maintained (to control durability, availability, and read performance). Data

Corbett et. al. Spanner: Google's Globally-Distributed Database. OSDI 2012

Next: Distributed Consensus

- AKA 6.5840 in 20 min
  - JK

- Why use consensus?
  - Standard way to achieve availability
  - Systems often pair this with 2PC to fix the issue where failed nodes block progress in the system
  - Mental model: 2PC, but each node is replicated with some consensus algorithm (e.g., Paxos)

Consensus

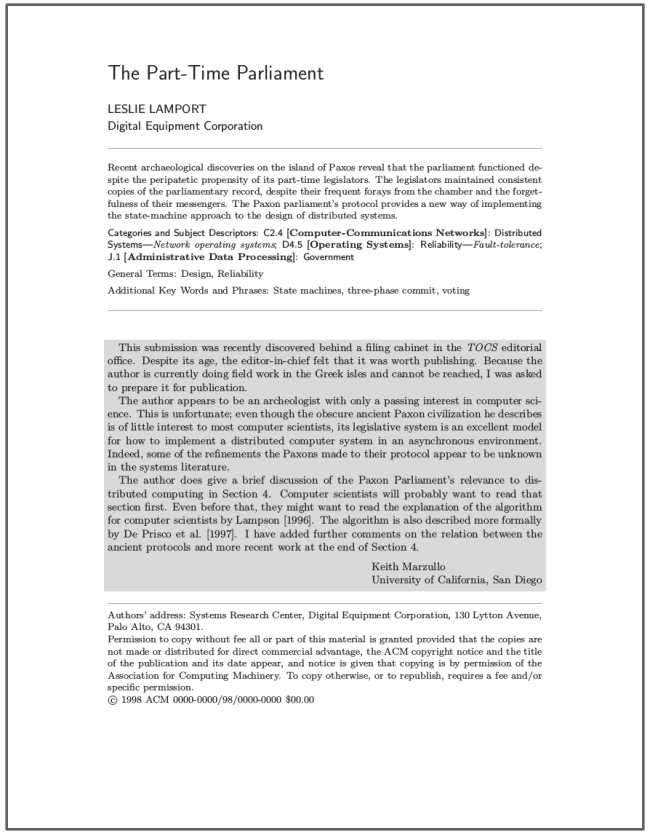
- Fundamental problem in distributed systems
  - Powerful primitive: many protocols can be easily implemented using consensus as a primitive
  - Recall: Impossible in general under asynchronous network with node failures
- Idea: can still have practical algorithms that compromise certain guarantees

Practical Consensus

- Compromise: guaranteed consensus validity, but only guaranteed termination if no failure over a stable period of time
  - Often reasonable assumption in practice
- First known protocol: Viewstamped Replication (from MIT)
- More well-known variants: Paxos, Raft

Paxos

- Key mechanism: *quorums*
  - Given a known set of nodes, a quorum is when at least half of the nodes agree on something
- Nice thing about quorums: any two quorums have non-zero intersection
  - Prevents split-brain situation from happening
- High level idea: nodes vote and accept result that has a majority
  - Many nuances, not in scope of this class



The Part-Time Parliament

LESLIE LAMPORT
Digital Equipment Corporation

Recent archaeological discoveries on the island of Paxos reveal that the parliament functioned despite the peripatetic propensity of its part-time legislators. The legislators maintained consistent copies of the parliamentary record, despite their frequent forays from the chamber and the forgetfulness of their messengers. The Paxon parliament's protocol provides a new way of implementing the state-machine approach to the design of distributed systems.

Categories and Subject Descriptors: C2.4 [**Computer-Communications Networks**]: Distributed Systems—*Network operating systems*; D4.5 [**Operating Systems**]: Reliability—*Fault-tolerance*; J.1 [**Administrative Data Processing**]: Government

General Terms: Design, Reliability

Additional Key Words and Phrases: State machines, three-phase commit, voting

This submission was recently discovered behind a filing cabinet in the *TOCS* editorial office. Despite its age, the editor-in-chief felt that it was worth publishing. Because the author is currently doing field work in the Greek isles and cannot be reached, I was asked to prepare it for publication.

The author appears to be an archeologist with only a passing interest in computer science. This is unfortunate; even though the obscure ancient Paxon civilization he describes is of little interest to most computer scientists, its legislative system is an excellent model for how to implement a distributed computer system in an asynchronous environment. Indeed, some of the refinements the Paxons made to their protocol appear to be unknown in the systems literature.

The author does give a brief discussion of the Paxon Parliament's relevance to distributed computing in Section 4. Computer scientists will probably want to read that section first. Even before that, they might want to read the explanation of the algorithm for computer scientists by Lampson [1996]. The algorithm is also described more formally by De Prisco et al. [1997]. I have added further comments on the relation between the ancient protocols and more recent work at the end of Section 4.

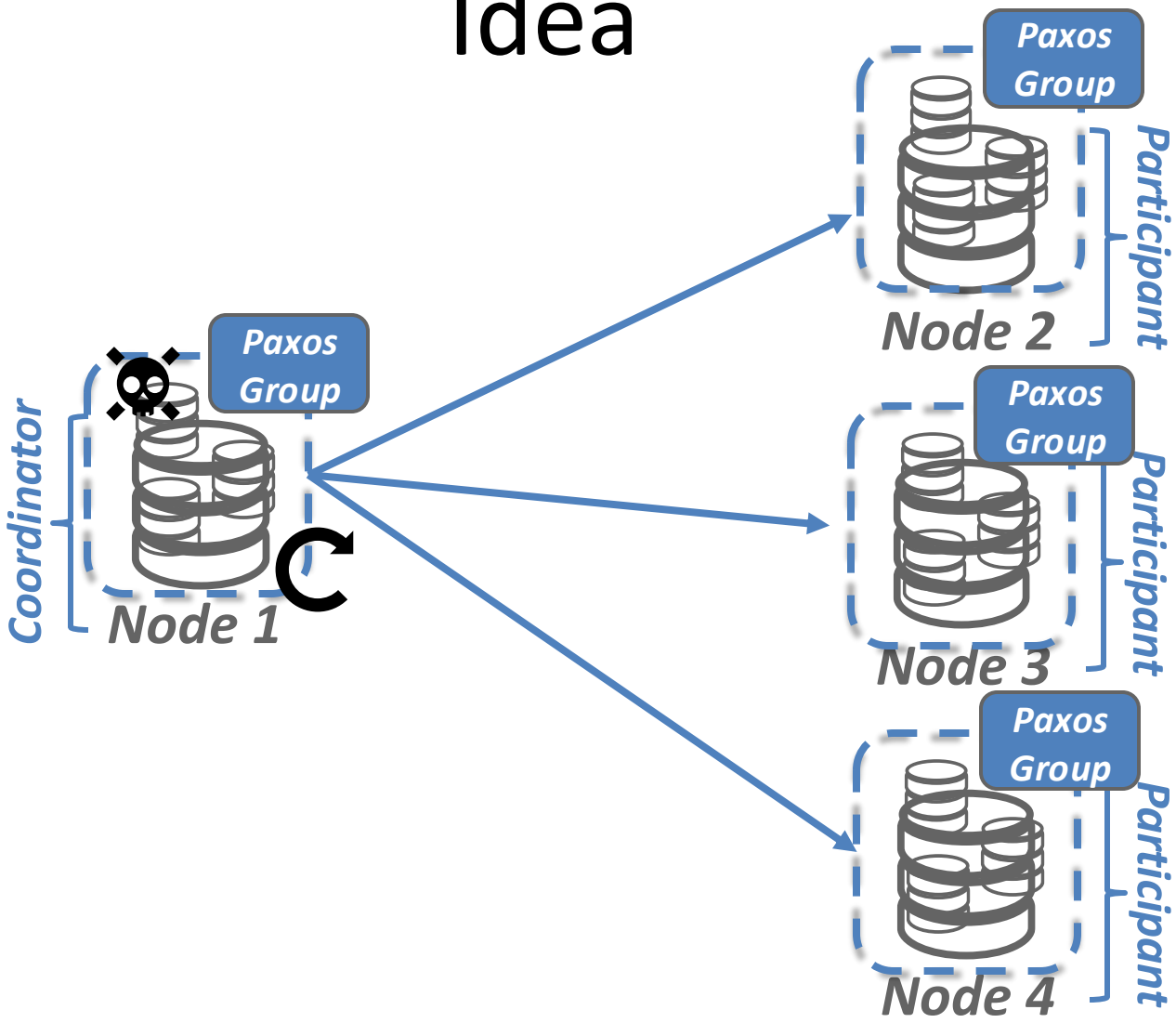
Keith Marzullo
University of California, San Diego

Authors' address: Systems Research Center, Digital Equipment Corporation, 130 Lytton Avenue, Palo Alto, CA 94301.

Mental Model

- Paxos/Raft can be used to provide the abstraction of a replicated state machine
  - Useful to think of the state machine as operating over a log of operations
  - Nodes can vote directly to append each entry or vote for a leader to append entries
  - Important: with consensus, a majority of nodes will safely and permanently record operations in the same order

Idea


Paxos Group
Coordinator
Node 1
Paxos Group
Participant
Node 2
Paxos Group
Participant
Node 3
Paxos Group
Participant
Node 4

Are we done?

- If we had simply replaced a database node with an RSM, performance would tank

- Idea: separation of data plane and control plane
  - Data is replicated through simple log shipping
  - Paxos for leader election, configuration changes, etc.

Are we done?

- What if the membership changes?
  - Membership can be modeled as yet another operation on RSM
- Potential optimizations: avoid disk writes on Paxos by kicking out crashed nodes and replenishing them with new nodes

Are we done?

- Recall examples:
  - Example 1: reserve a rental car and an airline flight, and only commit if both are available.
  - Example 2: transfer money from bank 1 to bank 2

- Paradoxically, 2PC is so slow and ill-behaved that neither of these use cases would typically consider distributed transactions!
  - Topic for another class…

Parting Thought

- Distributed systems are tricky
  - We can implement distributed transactions with strong guarantee, but at cost of performance
- We will discuss how systems can make the opposite trade-off: performance at the cost of strong guarantees next Thursday (4/16)
- Next class: Vector DB